

Guía Docente Diapositiva por Diapositiva — Sesión 1: Introducción, Virtualización y Primeros Contenedores

Curso: Docker desde Cero: Crea y Despliega Aplicaciones (10ma Edición 2026)

Instructor: Ing. Cristian Jampier Chileno Segundo | OTI - UNI

Programa: Programa de Iniciación Tecnológica (PIT 2026) — Universidad Nacional de Ingeniería

Total Diapositivas: 43 Diapositivas

Instrucciones de Orientación Pedagógica

Esta guía contiene la explicación detallada y el guión profesional en primera persona para abordar **cada una de las 43 diapositivas** de la presentación oficial de la Sesión 1. Está diseñada para guiar la clase paso a paso, garantizando que los estudiantes comprendan la teoría, sigan los comandos en la terminal y absorban los conceptos clave de la tecnología de contenedores.

Explicación Diapositiva por Diapositiva (1 a 43)

 Diapositiva 1: DOCKER DESDE CERO: Crea y Despliega Aplicaciones

Contenido de la PPT:

```
DOCKER DESDE CERO: Crea y Despliega Aplicaciones
INSTRUCTOR: Cristian Jampier Chileno Segundo
PROGRAMA DE INICIACIÓN TECNOLÓGICA – PIT 2026
Oficina de Tecnologías de la Información (OTI - UNI)
Programa Completo – PIT 2026
```

Guión del Docente (Lo que debes decir a los alumnos):

"Muy buenos días/tardes a todos. Bienvenidos al curso **Docker desde Cero: Crea y Despliega Aplicaciones**, correspondiente al Programa de Iniciación Tecnológica (PIT 2026) impartido por la Oficina de Tecnologías de la Información (OTI) de la Universidad Nacional de Ingeniería. Mi nombre es **Cristian Chileno Segundo** y seré su instructor durante este curso. Hoy iniciaremos nuestro camino en el mundo de los contenedores. Docker es una herramienta fundamental en la infraestructura y desarrollo moderno. En este curso no nos quedaremos solo en la teoría; nos enfocaremos en aprender practicando directamente en la terminal desde el primer día."

Acción en Consola / Pizarra:

- Proyectar la portada del curso en pantalla completa.
- Escribir en la pizarra los canales de consulta y la ruta del repositorio del curso:
<https://github.com/Crsitian22/docker-desde-cero-pit>.

💡 Tip de Gestión del Aula:

- Presentar brevemente las reglas de la clase: mantener el micrófono silenciado, usar la opción de levantar la mano para intervenir o escribir por el chat para resolver dudas en los puntos de pausa activa.

📄 Diapositiva 2: SESIÓN 1 — Índice del Temario

Contenido de la PPT:

SESIÓN 1

1. Introducción y Fundamentos
2. Virtualización e Hipervisores
3. Instalación de Docker
4. Imágenes y Contenedores
5. Comandos Esenciales
6. Primera Aplicación

👤 Guión del Docente (Lo que debes decir a los alumnos):

"En esta primera sesión abordaremos seis bloques estructurados progresivamente:

1. Primero, entenderemos los **Fundamentos** y la problemática clásica que originó la tecnología de contenedores.
2. Segundo, analizaremos cómo se compara Docker con la **Virtualización tradicional e Hipervisores**.
3. Tercero, revisaremos la ruta oficial de **Instalación de Docker** según su sistema operativo.
4. Cuarto, estableceremos la diferencia clara entre una **Imagen y un Contenedor**.
5. Quinto, ejecutaremos los **Comandos Esenciales** en la terminal.
6. Y finalmente, construiremos y ejecutaremos nuestra **Primera Aplicación Web en Flask** contenerizada."

👤 Acción en Consola / Pizarra:

- Marcar brevemente los hitos de la clase indicando que la segunda mitad de la sesión será 100% práctica en la terminal.

💡 Tip de Gestión del Aula:

- Preguntar a la clase: "*¿Cuántos de ustedes han trabajado antes con máquinas virtuales como VirtualBox o VMware?*" Esto te dará una métrica inicial de la experiencia técnica del grupo.

📄 Diapositiva 3: Objetivo de la Sesión 1

Contenido de la PPT:

Objetivo de la sesión 01:

- Explicar el concepto de contenerización.
- Explicar qué problema resuelve Docker.
- Ubicar Docker frente a VM e hipervisores.
- Identificar el flujo para instalar una VM base.
- Saber dónde instalar Docker y Compose desde la documentación oficial.
- Diferenciar imagen y contenedor.
- Ejecutar comandos básicos: run, ps, stop, rm.
- Construir y ejecutar una primera app en contenedor.

MENOS TEORÍA, MÁS TERMINAL

Guión del Docente (Lo que debes decir a los alumnos):

"Nuestra meta al finalizar esta clase es muy clara: al terminar la jornada, cada uno de ustedes sabrá exactamente qué es la contenerización, qué problema resuelve, cómo se diferencia de una máquina virtual, sabrá buscar la instalación en la documentación oficial, comprenderá el ciclo de vida de un contenedor y habrá construido y desplegado su propia aplicación web con un Dockerfile en su computadora. Nuestro lema en la OTI-UNI para este curso es: **Menos teoría, más terminal.**"

Acción en Consola / Pizarra:

- Anotar en un extremo de la pizarra los tres comandos principales que se dominarán hoy: `docker run`, `docker build`, `docker ps`.

Tip de Gestión del Aula:

- Remarcar a los estudiantes que no tengan miedo a los errores de terminal; cada error en pantalla es una oportunidad para aprender a diagnosticar.

Diapositiva 4: Bloque 1 — Introducción y Fundamentos

Contenido de la PPT:

Introducción y Fundamentos

Guión del Docente (Lo que debes decir a los alumnos):

"Iniciamos formalmente con el **Bloque 1: Introducción y Fundamentos**. Antes de escribir un solo comando, debemos entender por qué nació esta tecnología y cuál es la crisis real en el desarrollo de software que Docker vino a resolver en la industria."

Acción en Consola / Pizarra:

- Transición visual hacia el dilema del desarrollo de software.

Tip de Gestión del Aula:

- Captar la atención planteando la clásica frase: "¿A quién de ustedes le ha pasado que el código funciona en su laptop pero falla en la PC de su compañero o en el servidor?"

Diapositiva 5: El Problema: "En mi PC funciona"

Contenido de la PPT:

EL PROBLEMA: "En mi PC funciona"

- La app funciona en la laptop del desarrollador.
- En otro equipo falla por versiones distintas.
- En producción aparecen librerías faltantes o configuraciones diferentes.
- El equipo pierde tiempo reconstruyendo el entorno.

CLAVE: Docker busca que el entorno viaje junto con la aplicación.

Guión del Docente (Lo que debes decir a los alumnos):

"El problema clásico de la informática es la famosa frase: 'En mi máquina sí funciona'. Cuando desarrollamos software, nuestra laptop tiene instaladas versiones específicas de Python, bibliotecas, variables de entorno y parches del sistema operativo. Pero cuando enviamos ese código al servidor de pruebas o producción, el sistema falla porque le falta una librería, la versión de la base de datos es distinta o faltan permisos. El equipo pierde horas o días intentando adivinar qué falta en el servidor. La clave de Docker es simple: **hacer que el entorno de ejecución viaje empaquetado junto con el código de la aplicación.**"

Acción en Consola / Pizarra:

- Dibujar en la pizarra el esquema del problema tradicional: **Código (Laptop Dev) + Librerías Locales** ---> **Servidor Producción (¡ERROR! Faltan dependencias)**

Tip de Gestión del Aula:

- Hacer una pausa activa: "¿Por qué creen que reinstalar librerías a mano en producción es un riesgo para las empresas?" (Respuestas esperadas: fallas de seguridad, tiempo fuera de servicio, inconsistencia de versiones).

Diapositiva 6: Solución: La contenerización

Contenido de la PPT:

SOLUCIÓN: La contenerización

- Aislar la aplicación en un contenedor reproducible.
- Empaquetar código, dependencias y configuración en una imagen.
- Compartir esa imagen con el equipo o con un registro central.
- Ejecutarla en distintos entornos con menos diferencias.

RESULTADO: La app deja de depender de "lo que justo estaba instalado" en una máquina.

App + dependencias

Guión del Docente (Lo que debes decir a los alumnos):

"La solución tecnológica a este problema es la **contenerización**. En lugar de enviar únicamente los archivos fuente de texto de nuestro programa, creamos una unidad estandarizada llamada **Imagen** que empaqueta: el código, el runtime (por ejemplo Python o Node.js), las dependencias exactas y las configuraciones mínimas. Luego compartimos esa imagen mediante un registro central (como Docker Hub). Al desplegarla en cualquier computadora o servidor, este ejecutará exactamente la misma versión aislada. El resultado es que la aplicación deja de depender de 'lo que justo estaba instalado' en el servidor."

Acción en Consola / Pizarra:

- Dibujar la solución con Docker: **Código + Runtime + Dependencias = IMAGEN DOCKER ---> Corre idéntico en Laptop, Testing y Producción.**

Tip de Gestión del Aula:

- Explicar la analogía del contenedor marítimo de Malcolm McLean (1956): antes los barcos cargaban sacos y cajas sueltas; el contenedor metálico estandarizó el transporte marítimo mundial. Docker hace lo mismo con el software.

Diapositiva 7: ¿Qué es la contenerización?

Contenido de la PPT:

¿QUÉ ES LA CONTENERIZACIÓN?

- Aplicación + dependencias.
- Configuración mínima reproducible.
- Ejecución aislada del resto del sistema.
- Menos problemas de compatibilidad.

EN PALABRAS SIMPLES:

La contenerización empaqueta una aplicación con lo necesario para ejecutarla de forma uniforme en diferentes entornos.

Guión del Docente (Lo que debes decir a los alumnos):

"En palabras muy simples para su examen o para explicarlo a un colega: **La contenerización es la técnica de empaquetar una aplicación junto con sus dependencias y su configuración mínima para lograr una ejecución aislada, uniforme y reproducible en cualquier entorno.** Aislar la aplicación del resto del sistema operativo anfitrión evita que conflictos entre librerías afecten la ejecución de otros programas."

Acción en Consola / Pizarra:

- Enfatizar tres palabras clave en la pizarra: **Empaquetar, Aislar, Reproducir.**

Tip de Gestión del Aula:

- Pregunta rápida: "Si tengo una app en Python 2.7 y otra en Python 3.12, ¿pueden correr en la misma máquina con Docker sin chocar?" (Respuesta: Sí, porque cada una vive en su propio contenedor aislado).

Diapositiva 8: La pregunta natural: Entonces... ¿qué es Docker?

Contenido de la PPT:

Ya tenemos la idea. Ahora falta la herramienta.

LA PREGUNTA NATURAL: Entonces... ¿qué es Docker?

- Contenerizar es empaquetar una app con sus dependencias para ejecutarla igual en distintos entornos.
- Docker es una de las formas más usadas para construir, compartir y ejecutar esos contenedores.

Guión del Docente (Lo que debes decir a los alumnos):

"Ya tenemos clara la idea conceptual: contenerizar es el concepto general de empaquetado. Pero necesitamos la herramienta práctica que nos permita llevar esto a la realidad. Es aquí donde aparece **Docker**. Docker es la plataforma tecnológica y el conjunto de herramientas más utilizado en el mundo para **construir, compartir y ejecutar** esos contenedores de software de manera sencilla desde una línea de comandos."

Acción en Consola / Pizarra:

- Anotar la relación: **Concepto = Contenerización | Herramienta = Docker.**

Tip de Gestión del Aula:

- Aclarar que existen otras tecnologías de contenedores (como Podman o LXC), pero Docker es el estándar de la industria y la herramienta que aprenderemos a dominar.

Diapositiva 9: Entonces... qué es Docker

Contenido de la PPT:

ENTONCES... QUÉ ES DOCKER

Si una VM virtualiza una máquina...

Docker ejecuta aplicaciones en contenedores.

Guión del Docente (Lo que debes decir a los alumnos):

"Hagamos esta analogía directa: Mientras que una **Máquina Virtual (VM)** virtualiza una computadora física completa (con su propio sistema operativo, procesador virtual y disco), **Docker** únicamente virtualiza el entorno necesario para ejecutar aplicaciones dentro de procesos aislados sobre el mismo sistema operativo."

Acción en Consola / Pizarra:

- Escribir en la pizarra:
 - **Máquina Virtual** -> Virtualiza Hardware y Sistema Operativo Completo (Gigabytes).
 - **Contenedor Docker** -> Virtualiza el proceso de la Aplicación (Megabytes).

Tip de Gestión del Aula:

- Remarcar esta diapositiva como pregunta recurrente en entrevistas técnicas de DevOps y SysAdmins.
-

Diapositiva 10: Docker en una frase

Contenido de la PPT:

DOCKER EN UNA FRASE

DEFINICIÓN PRÁCTICA:

Docker permite empaquetar una aplicación con sus dependencias para ejecutarla de forma consistente en diferentes equipos.

Guión del Docente (Lo que debes decir a los alumnos):

"Si tuvieran que definir Docker en una sola frase durante una reunión técnica o examen, memoricen esta **Definición Práctica**: 'Docker permite empaquetar una aplicación con sus dependencias para ejecutarla de forma consistente y totalmente idéntica en cualquier equipo'. Eso es Docker en su esencia primaria."

Acción en Consola / Pizarra:

- Subrayar la palabra **CONSISTENTE**. Consistencia significa que si compila hoy en tu laptop, funcionará igual en el servidor mañana.

Tip de Gestión del Aula:

- Pedir a un alumno al azar que repita la definición con sus propias palabras para fijar el aprendizaje.
-

Diapositiva 11: Docker: Introducción Práctica

Contenido de la PPT:

DOCKER: Introducción Práctica

IDEA CENTRAL:

- Docker es una plataforma open source para crear, compartir y ejecutar contenedores.
- Implementa la idea de contenerización en el flujo diario de desarrollo.
- Facilita pruebas, despliegues y trabajo en equipo.
- Se apoya en imágenes, contenedores, registros y la CLI de Docker.
- Docker no reemplaza toda VM: resuelve la ejecución reproducible de aplicaciones.

👤 Guión del Docente (Lo que debes decir a los alumnos):

"Revisemos los pilares de la idea central de Docker:

1. Es una plataforma de código abierto (Open Source).
2. Se integra naturalmente en el flujo diario de programadores y administradores de sistemas.
3. Se sostiene sobre 4 componentes fundamentales que usaremos hoy: **Imágenes, Contenedores, Registros** y la **CLI (Línea de comandos)**.
4. Un punto muy importante: Docker **no** busca reemplazar a todas las máquinas virtuales del planeta; busca resolver la ejecución reproducible de aplicaciones web y servicios."

👤 Acción en Consola / Pizarra:

- Dibujar los 4 Pilares de Docker en la pizarra: [CLI] ----> [IMAGEN] ----> [CONTENEDOR] <--> [REGISTRO (Docker Hub)]

💡 Tip de Gestión del Aula:

- Verificar que todos los alumnos entiendan el gráfico de pilares antes de avanzar al temario de lo que se cubrirá en la clase.

📄 Diapositiva 12: Qué sí veremos y qué no veremos hoy

Contenido de la PPT:

Qué sí veremos y qué no veremos hoy

HOY SÍ:

- Concepto de Docker.
- Concepto de contenerización.
- Ruta oficial de instalación.
- Imagen vs contenedor.
- Comandos esenciales.
- Primera app con Dockerfile.

HOY NO:

- Kubernetes, OCI profundo o cgroups.
- YAML avanzado.
- Redes y volúmenes a detalle.
- Producción con Nginx.

👤 Guión del Docente (Lo que debes decir a los alumnos):

"Para gestionar adecuadamente las expectativas de la sesión de hoy: **Hoy SÍ aprenderemos y ejecutaremos:** los conceptos fundamentales, la guía de instalación oficial, la diferencia técnica entre imagen y contenedor, los comandos básicos de la CLI y crearemos nuestra primera app Flask con un Dockerfile real. **Hoy NO saturaremos la clase con:** Kubernetes, orquestadores complejos, redes multi-host avanzadas ni configuraciones de Nginx de producción. Esos temas están programados paso a paso para las sesiones 3, 4, 5 y 6 de nuestro curso."

Acción en Consola / Pizarra:

- Tranquilizar a los alumnos indicando que el curso es progresivo y nada se omitirá en la totalidad de las 6 sesiones.

Tip de Gestión del Aula:

- Recomendar a los alumnos centrarse 100% en los comandos básicos de hoy para tener bases sólidas.

Diapositiva 13: Pero... ¿cómo funciona?

Contenido de la PPT:

PERO... ¿CÓMO FUNCIONA?
Entendido... ¿Pero cómo funciona?
¿Es Docker simplemente otra... Máquina Virtual?

Guión del Docente (Lo que debes decir a los alumnos):

"Una vez entendido el beneficio, surge la pregunta técnica inevitable: *'Entendido profesor, pero por dentro... ¿cómo funciona realmente? ¿Es Docker simplemente otra máquina virtual con un logo bonito de una ballena?'* La respuesta es rotundamente **NO**. Para entender la diferencia profunda, entramos al Bloque 2 sobre Virtualización e Hipervisores."

Acción en Consola / Pizarra:

- Transición hacia la sección de comparación entre VM y Contenedores.

Tip de Gestión del Aula:

- Pausa de interacción: Preguntar si alguien sabe qué es el Kernel de un sistema operativo.

Diapositiva 14: Bloque 2 — Virtualización e Hipervisores

Contenido de la PPT:

Virtualización e Hipervisores

Guión del Docente (Lo que debes decir a los alumnos):

"Entramos al **Bloque 2: Virtualización e Hipervisores**. Vamos a analizar cómo se ejecutaban las aplicaciones antes de Docker y cómo la arquitectura del Kernel de Linux hizo posible la revolución de los contenedores."

Acción en Consola / Pizarra:

- Escribir como título en la pizarra: **Virtualización Clásica vs Contenerización.**

💡 Tip de Gestión del Aula:

- Mantener la atención de los estudiantes enfocándose en los aspectos de rendimiento y recursos (RAM/CPU).

📄 Diapositiva 15: Antes de Docker: máquinas virtuales

Contenido de la PPT:

Antes de Docker: máquinas virtuales

IDEA CLAVE:

- Una VM ejecuta un sistema operativo completo sobre hardware virtual.
- Permite aislar entornos: Linux, Windows Server, laboratorios, pruebas.
- Es útil cuando necesitas simular una máquina completa.
- Consume más recursos que un contenedor porque incluye su propio kernel.
- La VM virtualiza una máquina; Docker empaqueta procesos aislados.

👤 Guión del Docente (Lo que debes decir a los alumnos):

"Repasemos cómo funcionan las máquinas virtuales tradicionales: Una Máquina Virtual (VM) emula una computadora física completa mediante software. Para funcionar, requiere un sistema operativo invitado completo (Guest OS), con su propio Kernel, drivers, servicios de fondo y gestor de paquetes. Esto significa que si ejecutas 3 máquinas virtuales en tu servidor, estás ejecutando 3 sistemas operativos completos en paralelo, consumiendo gigabytes de disco y gigabytes de RAM solo en mantener vivos esos sistemas operativos. En cambio, Docker no emula hardware ni carga sistemas operativos completos; solo ejecuta **procesos aislados que comparten el mismo Kernel del sistema operativo anfitrión.**"

🏠 Acción en Consola / Pizarra:

- Dibujar la pila de una VM: [App] -> [Librerías] -> [Guest OS (Kernel Propio)] -> [Hipervisor] -> [Host OS] -> [Hardware]

💡 Tip de Gestión del Aula:

- Subrayar la diferencia: **Guest OS** (VM) vs **Kernel Compartido** (Contenedor).

📄 Diapositiva 16: Qué es un hipervisor

Contenido de la PPT:

QUÉ ES UN HIPERVISOR

PARA ESTE CURSO:

Si necesitas un laboratorio local, una VM con Linux puede servir como entorno limpio para instalar Docker y practicar sin afectar tu sistema principal.

Guión del Docente (Lo que debes decir a los alumnos):

"Un **Hipervisor** (o Monitor de Máquinas Virtuales) es la capa de software que permite crear y gestionar máquinas virtuales sobre el hardware físico. Existen dos tipos de hipervisores en la industria:

1. **Tipo 1 (Bare Metal):** Corre directamente sobre el hardware físico sin un SO intermedio (ejemplos: VMware ESXi, Proxmox VE, Microsoft Hyper-V Server).
2. **Tipo 2 (Hosted):** Corre como una aplicación sobre un sistema operativo anfitrión (ejemplos: VirtualBox, VMware Workstation).

Para este curso, si alguno de ustedes no desea instalar Docker directamente en su sistema operativo principal de la laptop, puede usar una VM con Linux (mediante VirtualBox) como un entorno de laboratorio totalmente limpio."

Acción en Consola / Pizarra:

- Escribir en la pizarra la clasificación de Hipervisores:
 - **Tipo 1 (Directo en Hardware):** Proxmox, ESXi.
 - **Tipo 2 (Sobre SO Host):** VirtualBox, VMware Workstation.

Tip de Gestión del Aula:

- Aclarar que Docker **no** es un hipervisor. Docker utiliza primitivas del kernel Linux (Namespaces y cgroups).

Diapositiva 17: La siguiente duda

Contenido de la PPT:

LA SIGUIENTE DUDA:
¿Y Docker? ¿Es un hipervisor? ¿De qué tipo?

Guión del Docente (Lo que debes decir a los alumnos):

"Surgirá esta duda típica en sus repasos: '¿Y Docker? ¿Es un hipervisor Tipo 1 o Tipo 2?' La respuesta correcta es: **Docker NO es un hipervisor de ningún tipo**. Docker es un motor de gestión de contenedores que utiliza virtualización a nivel de sistema operativo. No emula procesadores, memorias BIOS ni placas madre virtualizadas."

Acción en Consola / Pizarra:

- Tachar en la pizarra: **Docker != Hipervisor**. Escribir: **Docker = Motor de Contenedores (Aislamiento de procesos por Kernel)**.

Tip de Gestión del Aula:

- Asegurarse de que ningún alumno cometa el error conceptual de calificar a Docker como hipervisor en las evaluaciones.

Diapositiva 18: Docker vs Máquina Virtual

Contenido de la PPT:

DOCKER VS MÁQUINA VIRTUAL

DIFERENCIA CLAVE:

La VM incluye un sistema operativo completo (Guest OS). El contenedor comparte el kernel del host y solo incluye lo que la app necesita.

Guión del Docente (Lo que debes decir a los alumnos):

"Aquí está la **Diferencia Clave** que deben recordar siempre: La Máquina Virtual incluye obligatoriamente un sistema operativo invitado completo (Guest OS). El Contenedor de Docker **comparte el Kernel del host** y únicamente empaqueta el código fuente y las librerías específicas que la aplicación requiere para funcionar."

Acción en Consola / Pizarra:

- Dibujar lado a lado el esquema comparativo:
 - VM:** App | Libs | Guest OS | Hipervisor | Host OS | Hardware
 - Contenedor:** App | Libs | Docker Engine | Host OS (Kernel Compartido) | Hardware

Tip de Gestión del Aula:

- Hacer notar cómo la eliminación del Guest OS ahorra gigabytes de espacio y acelera el arranque.

Diapositiva 19: VM vs Contenedor: Decisión Rápida

Contenido de la PPT:

VM VS CONTENEDOR: DECISIÓN RÁPIDA

Criterio	Máquina virtual	Contenedor
Sistema operativo	Incluye SO completo	Comparte kernel del host
Arranque	Más lento	Rápido
Consumo	Mayor RAM y disco	Menor consumo
Uso común	Laboratorios o servidores completos	Apps reproducibles
Aislamiento	Fuerte (kernel propio)	Proceso aislado (kernel compartido)

Guión del Docente (Lo que debes decir a los alumnos):

"Revisemos esta tabla comparativa de **Decisión Rápida**:

- Sistema Operativo:** La VM incluye un SO completo; el contenedor comparte el kernel del Host.
- Tiempo de Arranque:** Una VM tarda minutos en encender (debe bootear el kernel); un contenedor enciende en **milisegundos** (solo arranca el proceso).

3. **Consumo de Recursos:** Una VM consume gigabytes de RAM y disco; un contenedor consume pocos megabytes.
4. **Uso recomendado:** Usamos VMs cuando necesitamos aislar entornos completos con kernels distintos (ej. probar un Windows sobre Linux). Usamos contenedores para empaquetar aplicaciones reproducibles y microservicios.
5. **Aislamiento:** La VM brinda aislamiento fuerte a nivel de hardware; el contenedor brinda aislamiento a nivel de proceso por Kernel."

Acción en Consola / Pizarra:

- Proyectar la tabla y enfatizar los tiempos de arranque (Minutos vs Milisegundos).

Tip de Gestión del Aula:

- Preguntar a la clase: "*Si tengo que desplegar 50 microservicios web, ¿qué me conviene usar?*" (Respuesta: Contenedores).

Diapositiva 20: Instalando una VM: Flujo Recomendado

Contenido de la PPT:

INSTALANDO UNA VM: FLUJO RECOMENDADO
CONFIGURACIÓN SUGERIDA:
2 CPU, 4 GB RAM, 25 GB disco, red NAT para empezar.
Si necesitas acceso desde otros equipos, usa bridge.

Guión del Docente (Lo que debes decir a los alumnos):

"Si alguno de ustedes prefiere trabajar el curso dentro de una máquina virtual Linux limpia en lugar de su sistema operativo nativo, les sugiero los siguientes recursos mínimos:

- **Procesador:** 2 vCPUs.
- **Memoria RAM:** 4 GB de RAM.
- **Disco Rígido:** 25 GB de almacenamiento.
- **Configuración de Red:** Usar modo **NAT** para que la máquina virtual tenga acceso a internet directamente sin conflictos en su red doméstica. Si van a acceder desde otras PCs físicas, cambien a modo **Bridge (Puente)**."

Acción en Consola / Pizarra:

- Anotar los recursos recomendados en la pizarra para los alumnos que usan VirtualBox.

Tip de Gestión del Aula:

- Indicar que los requisitos de recursos de Docker en la máquina host son modestos para las prácticas de hoy.

Diapositiva 21: Bloque 3 — Instalación de Docker

Contenido de la PPT:

Instalación de Docker

👤 Guión del Docente (Lo que debes decir a los alumnos):

"Ingresamos al **Bloque 3: Instalación de Docker**. Vamos a revisar los criterios oficiales y las recomendaciones para instalar la herramienta en Windows, macOS y Linux."

👤 Acción en Consola / Pizarra:

- Mostrar en pantalla el navegador web con la página oficial de Docker.

💡 Tip de Gestión del Aula:

- Recordar a los alumnos que nunca deben descargar instaladores de fuentes de terceros o blogs no oficiales.

📄 Diapositiva 22: Qué debes instalar

Contenido de la PPT:

QUÉ DEBES INSTALAR
CRITERIO DEL CURSO:
No memorices comandos de instalación: pueden cambiar por sistema operativo.
Usa siempre la guía oficial y valida según tu plataforma.

👤 Guión del Docente (Lo que debes decir a los alumnos):

"Atención con el **Criterio del Curso: No memoricen comandos de instalación de memoria**. Los comandos de paquetes (**apt**, **dnf**, **yum**, **brew**) cambian con frecuencia según las versiones de los sistemas operativos. La habilidad profesional clave que enseñamos en la UNI es aprender a acudir siempre a la **documentación oficial de Docker Docs** (docs.docker.com), seleccionar su sistema operativo exacto y seguir los pasos actualizados."

👤 Acción en Consola / Pizarra:

- Escribir la URL de referencia: <https://docs.docker.com>.

💡 Tip de Gestión del Aula:

- Hacer hincapié en la importancia de consultar documentación oficial actualizada frente a tutoriales obsoletos de internet.

📄 Diapositiva 23: Ruta oficial según tu sistema

Contenido de la PPT:

Ruta oficial según tu sistema

Windows y macOS:

- Instalar Docker Desktop.
- Incluye Docker Engine, CLI y Docker Compose.
- Revisar requisitos de WSL 2 en Windows.

Linux o VM Linux:

- Instalar Docker Engine según distribución.
- Revisar pasos post-instalación.
- Instalar Compose desde la guía oficial.

Enlaces: <https://docs.docker.com/desktop/> | <https://docs.docker.com/engine/install/>

Guión del Docente (Lo que debes decir a los alumnos):

"Veamos la ruta recomendada según el sistema operativo de su equipo: **Si usan Windows o macOS:** Deben descargar e instalar **Docker Desktop**. Esta aplicación empaqueta en un solo instalador el motor (Docker Engine), la consola (CLI) y Docker Compose. En Windows, asegúrense de tener activado el componente **WSL 2 (Windows Subsystem for Linux 2)**, que provee el kernel Linux nativo requerido.

Si usan Linux (Ubuntu, Debian, Fedora) o una VM Linux: Deben instalar **Docker Engine** siguiendo los pasos de su distribución en la guía oficial, configurar los permisos del grupo **docker** (pasos post-instalación) para no requerir **sudo** constante, e instalar el plugin de Docker Compose."

Acción en Consola / Pizarra:

- Mostrar en la terminal cómo verificar si el grupo docker tiene permisos en Linux: `groups $USER`.

Tip de Gestión del Aula:

- Asistir a los alumnos que tengan Windows verificando si la casilla 'Use WSL 2 instead of Hyper-V' quedó marcada en Docker Desktop.

Diapositiva 24: Docker Compose y Enlaces Oficiales

Contenido de la PPT:

DOCKER COMPOSE Y ENLACES OFICIALES

REGLA PRÁCTICA:

Si una guía externa contradice la documentación oficial, se prioriza Docker Docs.

Tema	Cuándo usarlo	Documentación
---	---	---
Docker Desktop	Windows/macOS o experiencia integrada	Docker Desktop Docs
Docker Engine	Linux nativo o VM Linux	Docker Engine Docs
Post-instalación Linux	Permisos, servicio y ajustes posteriores	Post-install Docs
Docker Compose	Aplicaciones con varios servicios	Compose Docs

👤 Guión del Docente (Lo que debes decir a los alumnos):

"Establecemos una **Regla Práctica** para el curso: *Si cualquier guía de un foro o blog de internet contradice la documentación oficial, SIEMPRE se prioriza la documentación oficial de Docker Docs*. En la tabla en pantalla resumen cuándo usar cada componente:

- **Docker Desktop:** Para desarrollo rápido e integrado en Windows/macOS.
- **Docker Engine:** Para servidores Linux y entornos nativos de producción.
- **Pasos Post-instalación:** Para configurar el demonio y permisos de usuario en Linux.
- **Docker Compose:** Para gestionar stacks con múltiples contenedores."

🏠 Acción en Consola / Pizarra:

- Resaltar la importancia de la regla de oro del SysAdmin: Consultar siempre la fuente oficial.

💡 Tip de Gestión del Aula:

- Preguntar si todos los alumnos tienen su consola abierta y lista para iniciar los laboratorios interactivos.

📄 Diapositiva 25: Bloque 4 — Imágenes y Contenedores

Contenido de la PPT:

Imágenes y contenedores

👤 Guión del Docente (Lo que debes decir a los alumnos):

"Ingresamos al **Bloque 4: Imágenes y Contenedores**. Esta es la base conceptual más importante antes de empezar a escribir comandos. Vamos a diferenciar con claridad estos dos términos."

🏠 Acción en Consola / Pizarra:

- Dividir la pizarra en dos columnas: **IMAGEN** vs **CONTENEDOR**.

💡 Tip de Gestión del Aula:

- Asegurar la máxima concentración del aula en este punto.

📄 Diapositiva 26: Imagen vs Contenedor

Contenido de la PPT:

IMAGEN VS CONTENEDOR
REGLA MENTAL:
La imagen es la receta; el contenedor es el plato servido y ejecutándose.

👤 Guión del Docente (Lo que debes decir a los alumnos):

"Grabemos esta **Regla Mental** indispensable: **La Imagen es la receta de cocina; el Contenedor es el plato de comida preparado, servido y listo para comer en la mesa.**

- La **Imagen** es una plantilla estática, inmutable y de solo lectura que contiene el sistema base, librerías y código. No consume procesador ni se ejecuta por sí misma.
- El **Contenedor** es la instancia viva en ejecución de esa imagen. Puedes crear 10 contenedores idénticos a partir de una sola imagen."

🏠 Acción en Consola / Pizarra:

- Escribir en la pizarra: **1 Receta (Imagen de Nginx) ==> Puede generar N Contenedores (Web1, Web2, Web3...)**

💡 Tip de Gestión del Aula:

- Pedir a los alumnos ejemplos de la vida real (ej. Clase vs Objeto en programación orientada a objetos, Molde vs Galleta).

📄 Diapositiva 27: Ciclo de vida básico

Contenido de la PPT:

CICLO DE VIDA BÁSICO
LO MÍNIMO PARA OPERAR:
Ejecutar, listar, detener, eliminar contenedores y revisar imágenes locales.

👤 Guión del Docente (Lo que debes decir a los alumnos):

"El **Ciclo de vida básico** de un contenedor consta de 5 estados y acciones operativas mínimas:

1. **Obtener / Descargar (Pull/Run):** Descargar la imagen desde Docker Hub.
2. **Ejecutar (Run):** Crear e iniciar la instancia viva del contenedor.
3. **Listar (PS):** Verificar el estado del contenedor en memoria.
4. **Detener (Stop):** Enviar una señal de apagado al contenedor.
5. **Eliminar (RM):** Borrar el contenedor detenido para liberar espacio. Dominar estas 5 operaciones es lo mínimo indispensable para operar Docker en el trabajo diario."

🏠 Acción en Consola / Pizarra:

- Dibujar el diagrama del ciclo de vida: **[Docker Hub] --pull--> [Imagen Local] --run--> [Contenedor Activo] --stop--> [Detenido] --rm--> [Eliminado]**

💡 Tip de Gestión del Aula:

- Explicar que al eliminar un contenedor no se elimina la imagen base.

📄 Diapositiva 28: Bloque 5 — Comandos Esenciales

Contenido de la PPT:

Comandos Esenciales

👤 Guión del Docente (Lo que debes decir a los alumnos):

"Llegó el momento de abrir nuestras terminales. Entramos al **Bloque 5: Comandos Esenciales**. A partir de esta diapositiva, todos los alumnos ejecutarán los laboratorios guiados en vivo junto a mí."

🖥️ Acción en Consola / Pizarra:

- Cambiar la proyección para mostrar la terminal de comandos (Prompt de PowerShell, Bash o WSL).

💡 Tip de Gestión del Aula:

- Indicar a los alumnos que acomoden sus pantallas compartidas: mitad pantalla con la videollamada y mitad pantalla con la terminal.

📄 Diapositiva 29: Laboratorio 1: Comprobar Docker**Contenido de la PPT:**

LABORATORIO 1: COMPROBAR DOCKER

QUÉ VALIDAMOS:

- La CLI responde.
- El Engine está activo.
- Docker puede descargar y ejecutar imágenes.

Comandos:

```
docker --version
```

```
docker info
```

```
docker run hello-world
```

👤 Guión del Docente (Lo que debes decir a los alumnos):

"Iniciamos el **Laboratorio 1: Comprobar Docker**. Vamos a validar tres aspectos del sistema:

1. Que el cliente de consola responda ejecutando: `docker --version`.
2. Que el motor demonio esté corriendo ejecutando: `docker info`.
3. Que Docker pueda descargar e iniciar imágenes ejecutando: `docker run hello-world`.

Ejecuten los comandos en su consola ahora. Observen cómo en `docker run hello-world`, al no encontrar la imagen localmente, la descarga automáticamente de Docker Hub, imprime el saludo en pantalla y el contenedor termina su tarea."

🖥️ Acción en Consola / Pizarra:

```
# Comandos del Laboratorio 1
docker --version
docker info
docker run hello-world
```

💡 Tip de Gestión del Aula:

- Señalar en la consola la línea `Unable to find image 'hello-world:latest' locally` para explicar el proceso automático de descarga (`pull`).

📄 Diapositiva 30: Laboratorio 2: Ejecutar Nginx

Contenido de la PPT:

```
LABORATORIO 2: EJECUTAR NGINX
QUÉ SIGNIFICA:
• --name: nombre del contenedor.
• -d: ejecuta en segundo plano.
• -p 8080:80: publica el puerto.
• nginx: imagen usada.

Comandos:
docker run --name web-demo -d -p 8080:80 nginx
docker ps
# Abrir en navegador: http://localhost:8080
```

👤 Guión del Docente (Lo que debes decir a los alumnos):

"Pasamos al **Laboratorio 2: Desplegar un Servidor Web Nginx**. Vamos a ejecutar un servidor de páginas web real en segundo plano. Analicemos cada flag de nuestro comando:

- `--name web-demo`: Asigna un nombre humano para identificar el contenedor.
- `-d` (detached): Ejecuta el contenedor en segundo plano, liberando la consola.
- `-p 8080:80`: Mapea el puerto 8080 de nuestra laptop al puerto 80 interno del contenedor.
- `nginx`: Es el nombre de la imagen oficial del servidor Nginx.

Ejecutemos: `docker run --name web-demo -d -p 8080:80 nginx`. Luego verifiquemos que está activo con `docker ps`. Finalmente, abran su navegador web en `http://localhost:8080` para ver la página de bienvenida de Nginx."

👤 Acción en Consola / Pizarra:

```
# Comandos del Laboratorio 2
docker run --name web-demo -d -p 8080:80 nginx
docker ps
```

- Proyectar el navegador abriendo `http://localhost:8080`.

💡 Tip de Gestión del Aula:

- Explicar la regla del puerto `-p HOST:CONTENEDOR`: El primer número es el puerto del Host (tu laptop), el segundo es el del contenedor.

📄 Diapositiva 31: Laboratorio 3: Detener y Limpiar

Contenido de la PPT:

LABORATORIO 3: DETENER Y LIMPIAR

BUENAS PRÁCTICAS:

- No acumules contenedores detenidos.
- Usa nombres claros en laboratorios.
- Verifica antes de borrar.

Comandos:

```
docker stop web-demo
docker ps -a
docker rm web-demo
docker images
```

👤 Guión del Docente (Lo que debes decir a los alumnos):

"En el **Laboratorio 3** aprenderemos la limpieza de recursos. Buenas prácticas: No dejen acumulados contenedores detenidos sin uso en sus computadoras. Sigamos la secuencia:

1. Detenemos el contenedor con `docker stop web-demo`.
2. Verificamos que ya no sale en `docker ps` pero sí en `docker ps -a` (contenedores detenidos).
3. Eliminamos el contenedor con `docker rm web-demo`.
4. Verificamos con `docker images` que la imagen de Nginx aún permanece guardada en disco para futuros usos."

👤 Acción en Consola / Pizarra:

```
# Comandos del Laboratorio 3
docker stop web-demo
docker ps -a
docker rm web-demo
docker images
```

💡 Tip de Gestión del Aula:

- Aclarar que `docker rm` falla si el contenedor aún está corriendo; primero se debe detener con `docker stop` (o usar `-f` para forzar).

Diapositiva 32: Mapa Rápido de Comandos

Contenido de la PPT:

```
MAPA RÁPIDO DE COMANDOS
| COMANDO | CUÁNDO USARLO | DOCUMENTACIÓN / EJEMPLO |
|---|---|---|
| docker run | Crea y ejecuta un contenedor | docker run nginx |
| docker ps | Lista contenedores activos | docker ps |
| docker ps -a | Lista todos los contenedores | docker ps -a |
| docker stop | Detiene un contenedor | docker stop web-demo |
| docker rm | Elimina contenedores detenidos | docker rm web-demo |
| docker images | Lista imágenes locales | docker images |
```

👤 Guión del Docente (Lo que debes decir a los alumnos):

"Aquí tienen la tabla resumen del **Mapa Rápido de Comandos** que acabamos de practicar. Tengan esta tabla a la mano para su estudio:

- **docker run**: Para crear y arrancar.
- **docker ps**: Para listar activos.
- **docker ps -a**: Para listar activos y detenidos.
- **docker stop**: Para detener.
- **docker rm**: Para borrar contenedores.
- **docker images**: Para revisar imágenes descargadas."

🏠 Acción en Consola / Pizarra:

- Dejar la tabla visible en pantalla durante 1 minuto para fijación visual.

💡 Tip de Gestión del Aula:

- Preguntar si hay alguna duda con el mapa rápido antes de pasar a la creación de nuestra propia imagen.

Diapositiva 33: Bloque 6 — Primera Aplicación

Contenido de la PPT:

Primera Aplicación

👤 Guión del Docente (Lo que debes decir a los alumnos):

"Ingresamos al último bloque de hoy: **Bloque 6: Primera Aplicación**. Hasta ahora hemos descargado imágenes hechas por otros (como Nginx o Hello-world). Ahora aprenderemos a construir nuestra propia imagen personalizada desde código fuente en Python Flask."

🏠 Acción en Consola / Pizarra:

- Mostrar en el explorador de archivos la carpeta del código `codigo/sesion1`.

💡 Tip de Gestión del Aula:

- Generar entusiasmo: *"Ahora crearemos nuestra propia imagen desde cero con nuestro nombre"*.

📄 Diapositiva 34: Proyecto Práctico de Hoy

Contenido de la PPT:

PROYECTO PRÁCTICO DE HOY

META:

Crear una app mínima, construir su imagen y verla funcionando desde el navegador.

👤 Guión del Docente (Lo que debes decir a los alumnos):

"Nuestra **Meta Práctica**: Vamos a revisar el código fuente de una aplicación Flask mínima, escribiremos la receta de su Dockerfile, construiremos la imagen con `docker build`, la ejecutaremos con `docker run` y accederemos a ella a través del navegador web."

👤 Acción en Consola / Pizarra:

- Explicar la meta en 4 pasos: `Código` -> `Dockerfile` -> `docker build` -> `docker run`.

💡 Tip de Gestión del Aula:

- Pedir a todos los alumnos que abran la carpeta del repositorio `codigo/sesion1`.

📄 Diapositiva 35: Paso 1: Crear Archivos del Proyecto

Contenido de la PPT:

PASO 1: CREAR ARCHIVOS DEL PROYECTO

ESTRUCTURA ESPERADA:

```
cd codigos/sesion1
```

Archivos ya preparados para la práctica:

`app.py`

`requirements.txt`

`Dockerfile`

Los archivos de la app están en `codigos/sesion1` junto al material de esta sesión.

👤 Guión del Docente (Lo que debes decir a los alumnos):

"Paso 1: Nos posicionamos en la carpeta de la práctica en la terminal: `cd codigo/sesion1`. Dentro de esta carpeta encontraremos tres archivos clave:

1. `app.py`: El código de la aplicación web en Python.
2. `requirements.txt`: Las librerías necesarias.

3. Dockerfile: La receta de construcción de nuestra imagen."

Acción en Consola / Pizarra:

```
cd codigo/sesion1
ls -la    # En Linux/macOS
dir       # En Windows PowerShell
```

Tip de Gestión del Aula:

- Verificar que todos los alumnos se hayan posicionado correctamente en el directorio con `cd`.

Diapositiva 36: Paso 2: Código de la App Flask

Contenido de la PPT:

PASO 2: CÓDIGO DE LA APP FLASK

Guión del Docente (Lo que debes decir a los alumnos):

"Paso 2: Inspeccionemos el archivo `app.py`. Es una aplicación web construida con el framework Flask de Python. El código define la ruta raíz `/` y devuelve un saludo HTML: `'<h1>Hola Docker desde el PIT 2026 - UNI</h1>'`. Configura la aplicación para escuchar en la dirección `0.0.0.0` y en el puerto `5000`."

Acción en Consola / Pizarra:

```
# Contenido de app.py
from flask import Flask

app = Flask(__name__)

@app.route('/')
def hola():
    return "<h1>Hola Docker desde el PIT 2026 - UNI</h1><p>Mi primera app
    containerizada con éxito.</p>"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

Tip de Gestión del Aula:

- Explicar por qué usamos `host='0.0.0.0'`: permite que Flask escuche peticiones provenientes de fuera del contenedor.

Diapositiva 37: Paso 3: Dependencias

Contenido de la PPT:

```
PASO 3: DEPENDENCIAS
# requirements.txt
flask==3.0.3
POR QUÉ DECLARARLAS:
El contenedor debe poder instalar lo necesario sin depender de lo que tenga la
laptop del alumno.
```

Guión del Docente (Lo que debes decir a los alumnos):

"Paso 3: Revisemos `requirements.txt`. Este archivo declara la versión exacta de la librería que necesita nuestra app: `flask==3.0.3`. **Por qué es vital declararlas:** El contenedor instalará esta versión aislada durante la compilación de la imagen. La laptop del estudiante o del servidor de producción no necesita tener Python ni Flask instalado previamente; todo ocurrirá dentro del contenedor."

Acción en Consola / Pizarra:

```
# requirements.txt
flask==3.0.3
```

Tip de Gestión del Aula:

- Reiterar que con Docker no es necesario instalar lenguajes de programación en la máquina host.

Diapositiva 38: Paso 4: Dockerfile Mínimo

Contenido de la PPT:

```
PASO 4: DOCKERFILE MÍNIMO
```

Guión del Docente (Lo que debes decir a los alumnos):

"Paso 4: Abrimos el archivo `Dockerfile`. Este archivo contiene la secuencia de instrucciones para construir la imagen:

- `FROM python:3.12-slim`: Usa la imagen base ligera de Python 3.12.
- `WORKDIR /app`: Crea y establece `/app` como directorio interno de trabajo.
- `COPY requirements.txt .`: Copia el archivo de requisitos al contenedor.
- `RUN pip install --no-cache-dir -r requirements.txt`: Instala las librerías necesarias durante el build.
- `COPY app.py .`: Copia el código fuente de nuestra aplicación.
- `EXPOSE 5000`: Documenta que la app usa el puerto 5000.

- **CMD** ["python", "app.py"]: Define el comando de inicio en tiempo de ejecución."

Acción en Consola / Pizarra:

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
```

Tip de Gestión del Aula:

- Explicar la diferencia crucial: **RUN** se ejecuta al construir la imagen; **CMD** se ejecuta al encender el contenedor.

Diapositiva 39: Paso 5: Construir la imagen

Contenido de la PPT:

PASO 5: CONSTRUIR LA IMAGEN
LECTURA DEL COMANDO:
-t mi-flask:v1 asigna nombre y versión a la imagen.
El punto final . indica que el contexto de build es la carpeta actual.

Comandos:
docker build -t mi-flask:v1 .
docker images

Guión del Docente (Lo que debes decir a los alumnos):

"Paso 5: Vamos a compilar nuestra imagen con el comando **docker build**. Leamos detalladamente el comando: **docker build -t mi-flask:v1 .**

- **-t mi-flask:v1**: El flag **-t** (tag) le da un nombre (**mi-flask**) y una versión (**v1**) a la imagen.
- **.** (el punto al final): Indica el **contexto de build**, es decir, que utilice los archivos y el Dockerfile que están en la carpeta actual.

Ejecuten el comando ahora. Verán cómo Docker descarga la base de Python, instala Flask y guarda la nueva imagen. Al finalizar, verifiquen con **docker images**."

Acción en Consola / Pizarra:

```
docker build -t mi-flask:v1 .
docker images
```

💡 Tip de Gestión del Aula:

- Recordar a los alumnos que NUNCA deben olvidar el punto `.` al final del comando `docker build`.
-

📄 Diapositiva 40: Paso 6: Ejecutar la aplicación

Contenido de la PPT:

PASO 6: EJECUTAR LA APLICACIÓN

RESULTADO ESPERADO:

El navegador debe mostrar: Hola Docker desde PIT 2026

Comandos:

```
docker run --name flask-demo -d -p 5000:5000 mi-flask:v1
```

```
docker ps
```

```
# Abrir: http://localhost:5000
```

👤 Guión del Docente (Lo que debes decir a los alumnos):

"Paso 6: Ha llegado el momento de encender nuestra aplicación contenerizada. Ejecutamos: `docker run --name flask-demo -d -p 5000:5000 mi-flask:v1`. Verificamos con `docker ps` que el contenedor `flask-demo` está activo. Abran su navegador web e ingresen a: `http://localhost:5000`. ¡Felicitaciones! En pantalla deben ver el mensaje: **Hola Docker desde el PIT 2026 - UNI.**"

🏠 Acción en Consola / Pizarra:

```
docker run --name flask-demo -d -p 5000:5000 mi-flask:v1
docker ps
```

- Mostrar la ventana del navegador renderizando la app web en el puerto 5000.

💡 Tip de Gestión del Aula:

- Celebrar el logro con la clase: han completado el ciclo completo de creación y despliegue de una app contenerizada.
-

📄 Diapositiva 41: Paso 7: Limpiar el laboratorio

Contenido de la PPT:

PASO 7: LIMPIAR EL LABORATORIO

RESULTADO ESPERADO:

Primero detienes el contenedor, luego lo eliminas. La imagen queda disponible hasta que decidas borrarla.

Comandos:

```
docker stop flask-demo
docker rm flask-demo
# Opcional: borrar la imagen
docker rmi mi-flask:v1
```

👤 Guión del Docente (Lo que debes decir a los alumnos):

"Paso 7: Concluimos con la limpieza del laboratorio. La secuencia correcta es:

1. Detener el contenedor: `docker stop flask-demo`.
2. Eliminar el contenedor: `docker rm flask-demo`.
3. (Opcional) Borrar la imagen guardada: `docker rmi mi-flask:v1`. Noten que al eliminar el contenedor, la imagen sigue guardada en su disco duro para cuando quieran volver a usarla, a menos que ejecuten `docker rmi`."

👤 Acción en Consola / Pizarra:

```
docker stop flask-demo
docker rm flask-demo
docker rmi mi-flask:v1
```

💡 Tip de Gestión del Aula:

- Verificar que las consolas de los alumnos hayan quedado limpias.

📄 Diapositiva 42: Mapa Rápido de Comandos / Resolución de Errores

Contenido de la PPT:

```
MAPA RÁPIDO DE COMANDOS / RESOLUCIÓN DE ERRORES
| ERROR | CAUSA PROBABLE | SOLUCIÓN RÁPIDA |
|---|---|---|
| Puerto ocupado | Otro proceso usa 5000 | Cambiar a -p 5001:5000 |
| No abre en navegador | Contenedor detenido | Revisar docker ps -a |
| Imagen no encontrada | Nombre mal escrito | Revisar docker images |
| Build falla | Archivo faltante o typo | Verificar carpeta y Dockerfile |
```

👤 Guión del Docente (Lo que debes decir a los alumnos):

"Revisemos esta tabla de **Resolución de Errores Frecuentes**:

1. **Puerto ocupado:** Si les da error de puerto, es porque otra app usa el 5000. Solución: Cambiar el puerto del host, por ejemplo `-p 5001:5000`.
2. **No abre en navegador:** El contenedor se cayó. Solución: Ejecutar `docker ps -a` y revisar los logs con `docker logs flask-demo`.

3. **Imagen no encontrada:** Escribieron mal el nombre o versión. Solución: Verificar con `docker images`.

4. **Build falla:** Se olvidaron del punto final o hay un error de sintaxis en el Dockerfile."

Acción en Consola / Pizarra:

- Dejar la tabla proyectada para consulta de la clase.

Tip de Gestión del Aula:

- Responder dudas finales sobre errores experimentados durante el ejercicio.

Diapositiva 43: Checklist de Aprendizaje — Sesión 1

Contenido de la PPT:

CHECKLIST DE APRENDIZAJE – SESIÓN 1

- ✓ Puedo explicar qué es un hipervisor y para qué sirve una VM.
- ✓ Puedo describir el flujo básico para instalar una VM de laboratorio.
- ✓ Puedo ubicar la documentación oficial para instalar Docker y Compose.
- ✓ Puedo explicar qué es la contenerización y qué problema resuelve.
- ✓ Puedo explicar por qué Docker evita diferencias entre entornos.
- ✓ Puedo diferenciar imagen y contenedor con un ejemplo.
- ✓ Puedo ejecutar, listar, detener y eliminar contenedores.
- ✓ Puedo construir una imagen propia con `docker build`.
- ✓ Puedo ejecutar mi primera app usando `docker run -p`.

Guión del Docente (Lo que debes decir a los alumnos):

"Hemos llegado al final de nuestra primera sesión. Revisemos nuestro **Checklist de Aprendizaje**: Hoy han aprendido la teoría de hipervisores y contenerización, han conocido la documentación oficial de instalación, diferencian perfectamente una imagen de un contenedor, dominan los comandos `run`, `ps`, `stop`, `rm`, han compilado una imagen con `docker build` y han desplegado su primera aplicación web en la puerto 5000.

¡Excelente trabajo a todos! En la próxima sesión aprenderemos a escribir Dockerfiles profesionales de producción, optimización de capas y publicación en Docker Hub. ¡Nos vemos en la Sesión 2!"

Acción en Consola / Pizarra:

- Despedida de la sesión, recordar revisar el repositorio de GitHub y el cuestionario de evaluación de la Sesión 1.

Tip de Gestión del Aula:

- Recordar a los alumnos resolver la evaluación de 12 preguntas de la Sesión 1 disponible en el aula virtual.